SIMPLE & NATURAL VIDEO DEMONSTRATION SCRIPT (5 to 8 Mins)

Student: **Yash Janardan Sabale** (PRN: 2310423T26-27) | MITAOE Pune | DevOps IA Activity 1

Part 1: Self Introduction & Project Goal (0:00 – 1:00)

■■ WHAT TO SHOW ON SCREEN: Show Page 1 of Word Report with Name, PRN, and College details.

■■ *SPOKEN SCRIPT:*

"Hello everyone. My name is Yash Janardan Sabale, PRN 2310423T26-27, studying Computer Engineering at MIT Academy of Engineering, Pune. Today, I am presenting my practical demonstration for Internal Assessment Activity No. 1 in DevOps. My project is about automating the deployment of a real-world application called ShalaConnect. ShalaConnect is a school governance platform for Cluster Resource Centre Heads. Technically, it is a 3-tier enterprise application built using Java 21, Spring Boot 3.2.3, and PostgreSQL 15. Earlier, developers had to manually SSH into the server, install Java, configure the database, and run the JAR file in the background. This manual process took almost an hour and caused frequent errors and downtime. In this project, I used Ansible to automate the entire setup so that the server can be provisioned in under two minutes with zero errors."

Part 2: DevOps Lifecycle & Cloud Architecture Diagrams (1:00 – 2:30)

■■ WHAT TO SHOW ON SCREEN: Scroll down in Word report to Section 2 showing Figure 1, Figure 2, and Figure 3.

■■ *SPOKEN SCRIPT:*

"First, let us look at our DevOps Lifecycle in Figure 1. We designed an automated 8-stage pipeline: We Plan features in Jira, We Code using Java 21 with Git, We Build the package using Maven, We Test with automated unit tests, We Release the code to GitHub, We Deploy automatically using Ansible, We Operate the service using Linux systemd, and We Monitor health using our Spring Boot /api/health endpoint. Now look at Figure 2 and Figure 3, which show our AWS Cloud architecture. We used two separate EC2 virtual machines: 1. The first one is the Ansible Control Node at Public IP 13.201.133.251. This machine runs Ansible and holds our playbooks. 2. The second one is our Managed App Server at Public IP 13.126.181.31. The best part about Ansible is that it is agentless. We do not need to install any agent software on the target server. Ansible connects securely over standard SSH on port 22 and configures everything automatically."

Part 3: The 12 Live Execution Screenshots in the Report (2:30 – 4:00)

■■ WHAT TO SHOW ON SCREEN: Scroll through Section 3 of Word report showing the 12 screenshots.

■■ *SPOKEN SCRIPT:*

"Next, in Section 3 of my report, you can see the 12 authentic screenshots taken during the live deployment: Evidence 1 and 2 show the AWS Console where I launched the Control instance. Evidence 3 and 4 show the second instance, which is our target application server. Evidence 5 shows me connecting from my Windows PowerShell terminal into the Control Node using my private key shalaconnect-key.pem. Evidence 6 shows me checking the inventory file on the server. Evidence 7 shows the Ansible Ping test. Running ansible all -m ping returned ok: [ec2-app-server], confirming that both instances are communicating properly over SSH. Evidence 8 and 9 show the main playbook execution. You can clearly see my name, Yash Janardan Sabale, and my PRN in the initialization banner. The final summary shows 37 tasks OK and 0 failed. Evidence 10 shows systemctl status shalaconnect.service. The Java backend is active and running under systemd on port 8080. Evidence 11 shows the PostgreSQL database where all 14 application tables, like users, schools, and attendance, were created automatically. And Evidence 12 shows a curl command testing our public health endpoint, which returned HTTP 200 OK."

Part 4: Codebase Walkthrough on GitHub (4:00 – 6:30)

■■ WHAT TO SHOW ON SCREEN: Switch tabs to GitHub (YashSabale01/ShalaConnect/tree/main/DevOps_Ansible_Assignment). Click open each file.

■■ *SPOKEN SCRIPT:*

"Now let us switch to my GitHub repository and look at the actual Ansible code files that made this possible. 1. ansible.cfg: This is the main configuration file for Ansible. Here, we tell Ansible: The default login user is ec2-user. Privilege escalation is enabled with become = True and become_method = sudo, so Ansible can run root commands. We disabled host_key_checking so SSH runs automatically without prompting for confirmation. We enabled SSH multiplexing to make deployments 4 times faster. 2. inventory/inventory_aws_ec2.ini: An inventory file defines which servers Ansible will manage. Here, we defined ec2-app-server with IP 13.126.181.31. We gave the path to our private SSH key shalaconnect-key.pem. We grouped our server under [app_servers] and [webservers]. We defined common variables like app_port=8080 and nginx_port=80. 3. playbooks/site.yml: This is our master playbook. Instead of writing 100 lines in one file, this playbook cleanly calls our custom Ansible role named springboot_app. 4. The Role Tasks (roles/springboot_app/tasks/): Inside the role, the automation is split into 5 clean, modular task files: setup_environment.yml: This installs Java 21 Amazon Corretto, Maven, Nginx, and PostgreSQL 15. It also automatically initializes the database, enables local trust login, creates the school_management database, and creates a secure Linux system user called appuser. deploy_code.yml: This creates the directory /opt/shalaconnect-app, copies our Spring Boot JAR file from the Control Node to the App Server, and deploys our application-prod.properties file. configure_service.yml: This sets up the Linux systemd service (shalaconnect.service). If the Java process ever crashes or the server restarts, systemd will automatically restart the backend in 5 seconds. configure_nginx.yml: This configures Nginx as a reverse proxy. It listens on standard public Port 80 and safely forwards requests to Spring Boot on internal Port 8080. verify_health.yml: This performs an automated smoke test by calling the /api/health endpoint to make sure everything is working before declaring success."

Part 5: Live Public Telemetry Verification & Conclusion (6:30 – 7:30)

■■ WHAT TO SHOW ON SCREEN: Switch to the browser tab showing http://13.126.181.31/api/health and press Refresh (F5).

■■ *SPOKEN SCRIPT:*

"Finally, let us verify the live application running in production. In my browser address bar, you can see our public IP: http://13.126.181.31/api/health. When I refresh, the request hits Nginx on port 80, forwards to our Spring Boot backend on port 8080, checks the PostgreSQL database, and instantly returns: HTTP 200 OK with JSON response: status: UP, service: ShalaConnect API. In conclusion, this project successfully analyzed the DevOps lifecycle of ShalaConnect, designed a multi-node AWS cloud architecture, built an idempotent agentless Ansible role, deployed Java 21, PostgreSQL 15, Spring Boot, and Nginx automatically, and verified the live deployment with 12 authentic screenshots and public health telemetry. All code, documentation, and reports are committed to GitHub and ready for evaluation. Thank you very much!"